NLP Technical Report

Multi-Hop Question Answering Retrieval-Augmented Generation Methods Benchmark

Authors: Alexandru Profir, Tudor-Octavian Mihăiţă
Group: ICA 246-2

June 1, 2026

1 Problem Statement

1.1 Task definition: Open-Domain Multi-Hop Question Answering

[Author: Alexandru Profir]

Large Language Models (LLMs) have demonstrated to be a powerful tool for Natural Language Processing (NLP) tasks, especially the ones that require extensive levels of context reasoning, such as Question Answering (QA). However, their inherent factual knowledge is limited to what has been encoded in the parametric memory during training and cannot be reliably updated at inference time. As a result, LLMs may generate factually incorrect answers, a phenomenon referred to as **hallucination**. These limitations motivate the use of external knowledge sources that can be accessed dynamically when answering questions.

One prominent task is **Open-Domain Question Answering**, where a system must answer a natural language question by retrieving information from a large textual corpus rather than relying solely on its internal parameter weights. Formally, given a question q and a corpus of passages $\mathcal{C} = p_1, \ldots, p_N$, the goal is to generate an answer a. Solving this task requires two key capabilities: retrieving a set of relevant supported passages $\mathcal{S} \subseteq \mathcal{C}$ and generating the answer from the information contained in those passages. The task is considered **open-domain** because the supporting passages are not provided as input with the question and must be identified from a potentially very large corpus, containing various general knowledge.

This project focuses on the more challenging **multi-hop** setting of the QA task, where answering a question requires combining information from multiple sources. In this case, $|\mathcal{S}| \geq 2$, and the final answer can only be obtained through a sequence of reasoning steps. Compared to single-hop QA, multi-hop QA introduces additional retrieval challenges. Information required in later reasoning steps may have little lexical overlap with the original query, making it difficult to retrieve using standard similarity-based methods. Consequently, effective multi-hop QA requires not only accurate retrieval and answer generation, but also the ability to iteratively discover and connect relevant pieces of evidence across multiple documents.

The task therefore combines three core NLP problems: *information retrieval*, *answer generation*, and *multi-step reasoning*. Modern Retrieval-Augmented Generation (RAG) systems address these components within a unified pipeline, making open-domain multi-hop Question Answering a natural benchmark for evaluating advanced retrieval strategies.

1.2 The role of Retrieval-Augmented Generation

[Author: Tudor-Octavian Mihăiță]

Retrieval-Augmented Generation (RAG) has emerged as one of the most effective approaches for open-domain Question Answering [LPP+20]. A RAG system combines an external information retrieval component with a LLM: given a question, the retriever first selects a small set of relevant passages from an external corpus, and the language model then generates an answer conditioned on the retrieved evidence. This design extends the model's accessible knowledge beyond what is stored in its parameters while allowing answers to be grounded in explicit supporting information.

The initial implementation of RAG, often referred to as *Naive* RAG, retrieves the top-k passages most similar to the input question and provides them directly to the generator. While this approach performs well on many single-hop QA tasks, it becomes less effective in multi-hop settings. Information required for later reasoning steps may not be directly related to the original query, making it difficult for a single retrieval step to identify all relevant evidence.

To address this limitation, recent work has proposed a variety of *retrieval-side enhancements* that improve how evidence is selected and accumulated before generation. The approaches we choose to study are **cross-encoder re-ranking** [NC19], and **query decomposition** [KTF+22], including iterative retrieval-reasoning strategies [TBKS23, PZM+23]. Unlike approaches that modify or fine-tune the language model itself, these methods aim to improve performance by refining the retrieval process.

This project investigates the effectiveness of such retrieval-side techniques on multi-hop Question Answering. The evaluation is conducted on the **MuSiQue benchmark** [TBKS22], which was specifically designed to require reasoning across multiple pieces of evidence while reducing shortcuts that allow questions to be solved through simple retrieval heuristics. As a result, MuSiQue provides a challenging testbed for assessing the impact of retrieval quality on downstream question answering performance.

The core hypothesis of this project is that naive dense retrieval performs reasonably well on questions requiring a small number of reasoning steps, but its effectiveness decreases as the number of hops increases. We further investigate whether external retrieval enhancements can mitigate this degradation. Since none of the evaluated methods require additional training of either the retriever or the language model, the comparison isolates the effect of retrieval strategy itself, allowing a more direct assessment of its contribution to multi-hop QA performance.

1.3 Project Scope and Contributions

The project's scope is the design, implementation, and evaluation of a Retrieval-Augmented Generation benchmark for open-domain multi-hop QA. The system includes four retrieval methods (Classic RAG, Re-ranking, Static Decomposition, and Iterative Decomposition), a No-RAG baseline, an offline evaluation framework based on the MuSiQue dataset, and a Streamlit application for interactive analysis. Development was carried out by a two-person team, with each member taking ownership of one of the two main retrieval-side improvements and related infrastructure.

***Alexandru Profir*: Re-ranking, retrieval infrastructure, evaluation, and benchmarking interface**.

- Designed and implemented the **retrieve-then-rerank architecture** using dense retrieval followed by cross-encoder scoring with `BAAI/bge-reranker-base`.

- Developed the **retrieval backbone**, including the common retriever interface, naive retrieval baseline, and retrieval trace infrastructure.
- Built the **data preparation and indexing pipeline**, including MuSiQue sampling, corpus construction, deduplication, embedding generation, and ChromaDB indexing.
- Implemented the **evaluation framework** and retrieval/generation metrics used throughout the experiments.
- Developed the **Benchmark mode** of the Streamlit application, including gold-passage visualization and per-question evaluation.

***Tudor-Octavian Mihăiţă*: Query decomposition, generation pipeline, and interactive interface**.

- Designed and implemented the **Static** and **Iterative Query Decomposition** methods, including bridge-entity resolution through intermediate-answer generation and query rewriting.
- Developed the **generator abstraction layer**, supporting both Ollama and OpenAI-compatible backends through a unified interface.
- Designed the **prompting strategy** used for retrieval-augmented generation, decomposition, rewriting, and intermediate reasoning.
- Implemented the **RAG pipeline orchestration** and shared method factory used by both the evaluation framework and the application.
- Developed the **Chat** and **Compare** modes of the Streamlit application, together with the result aggregation and analysis utilities used for reporting perfromance metrics.

2 Proposed Solution

This section presents the proposed solution to the multi-hop Question Answering task introduced in Section 1. We first introduce the theoretical foundations underlying the implemented methods, including the Retrieval-Augmented Generation (RAG) paradigm and the retrieval-side techniques investigated in this project. We then describe the MuSiQue benchmark used for evaluation and the strategy employed to construct the retrieval corpus. Finally, we provide an overview of the application architecture developed to implement, execute, and compare the proposed methods in a unified framework.

2.1 Theoretical aspects

This subsection introduces the theoretical concepts underlying the methods implemented in this project. We first discuss the limitations of large language models as standalone QA systems and motivate the need for external knowledge retrieval. We then present the Retrieval-Augmented Generation (RAG) paradigm and describe the retrieval-side techniques evaluated in this work, namely cross-encoder re-ranking and query decomposition.

2.1.1 Large Language Models and parametric memory sectionauthorTudor-Octavian Mihăiţă

Large Language Models acquire factual knowledge during training and store it implicitly in their model parameters. This internal knowledge source is commonly referred to as *parametric*

memory. While parametric memory enables language models to answer a wide range of factual questions without external resources, it has important limitations in open-domain settings.

First, the knowledge encoded in the model is fixed after training and cannot be updated without additional fine-tuning. Second, factual coverage depends on the distribution of the training data, meaning that rare, recent, or domain-specific information may be poorly represented. Finally, language models may generate factually incorrect responses despite appearing confident, a phenomenon known as *hallucination*. These issues become particularly pronounced in multi-hop Question Answering, where answering a question often requires combining information from multiple sources rather than recalling a single fact.

In addition, the limited context window of a language model prevents entire document collections from being provided as input. Consequently, systems must identify and supply only the most relevant pieces of information. Retrieval-Augmented Generation addresses these limitations by coupling language models with an external retrieval mechanism, allowing answers to be grounded in dynamically retrieved evidence.

2.1.2 The RAG paradigm and Naive Retrieval

[Author: Alexandru Profir]

Retrieval-Augmented Generation (RAG) [LPP+20] combines a language model with an external document corpus. Given a question q, a retriever first selects a set of relevant passages $\mathcal{R}(q)$ from a corpus $\mathcal{C}$, after which a generator produces an answer conditioned on both the question and the retrieved evidence. The resulting answer distribution is given by Equation 1.

$$p(a \mid q) = p_\theta(a \mid q, \mathcal{R}(q)) \tag{1}$$

A key advantage of this architecture is the separation between retrieval and generation. The generator remains unchanged regardless of how passages are selected, allowing retrieval strategies to be studied independently. In this project, the language model and document corpus remain fixed, while only the retrieval component is modified.

The simplest retrieval strategy, commonly referred to as *Naive* RAG, relies on dense vector similarity. Following the dense retrieval paradigm introduced by **Dense Passage Retrieval** (DPR) [KOM+20], each passage $p_i \in \mathcal{C}$ is encoded into an embedding vector $\mathbf{e}_i$, while the input question is encoded into a query vector $\mathbf{q}$. Retrieval then consists of selecting the top-k passages with the highest cosine similarity to the query, as shown in Equation 2.

$$\mathcal{R}_{\text{naive}}(q) = \underset{p_i \in \mathcal{C}}{\text{top-k}} \ \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \, \|\mathbf{e}_i\|}. \tag{2}$$

This approach is efficient because **document embeddings can be precomputed and stored in an index**, leaving only query encoding and similarity search to be performed at inference time. Consequently, dense retrieval has become the standard baseline for open-domain question answering systems.

However, naive retrieval exhibits two important limitations. First, cosine similarity alone may fail to distinguish between several highly similar candidate passages, leading to suboptimal ranking of the retrieved evidence. Second, multi-hop questions often require retrieving information that is not directly related to the wording of the original query, making it difficult for a single retrieval step to identify all necessary evidence. These limitations motivate the two retrieval-side enhancements evaluated in this project: cross-encoder re-ranking and query decomposition.

2.1.3 Cross-encoder re-ranking

[Author: Alexandru Profir]

Cross-encoder re-ranking addresses one of the main limitations of dense retrieval: the inability of bi-encoders to model fine-grained interactions between a query and a candidate passage. Unlike bi-encoders, which encode queries and passages independently, a cross-encoder processes them jointly and directly predicts a relevance score. This scoring function is formalized in Equation 3, where ψ denotes the cross-encoder model and $s(q, p)$ represents the estimated relevance of passage p to query q. Because the query and passage are processed together, the model can exploit token-level interactions through the transformer's attention mechanism, generally producing more accurate relevance estimates than cosine similarity alone [NC19, RG19].

$$s(q, p) = \psi(q, p) \in \mathbb{R}. \tag{3}$$

The improved ranking quality comes at a significant computational cost. Since the relevance score depends jointly on both inputs, passage representations cannot be precomputed and stored in an index. Running a cross-encoder over an entire corpus is therefore impractical for large-scale retrieval tasks.

To balance efficiency and accuracy, modern retrieval systems employ a two-stage **retrieve-then-rerank** pipeline [NC19]. A bi-encoder first retrieves a larger candidate set of passages, after which the cross-encoder re-scores and re-orders these candidates. The overall procedure is summarized in Algorithm 1.

Algorithm 1 Cross-encoder re-ranking algorithm

Input: Question q, corpus index $\mathcal{C}$, bi-encoder ϕ, cross-encoder ψ, candidate size k_{cand}, final size k

Output: Top-k passages

$\mathbf{q} \leftarrow \phi(q)$

$\mathcal{P}_{\text{cand}} \leftarrow \text{top-k}_{\text{cand}\,p \in \mathcal{C}}\ \cos(\mathbf{q}, \phi(p))$ ▷ first-stage dense retrieval

for $p \in \mathcal{P}_{\text{cand}}$ **do**

  $s_p \leftarrow \psi(q, p)$ ▷ joint scoring of each candidate

end for

$\mathcal{P}_{\text{final}} \leftarrow$ top-k passages from $\mathcal{P}_{\text{cand}}$ ranked by s_p ▷ re-rank

return $\mathcal{P}_{\text{final}}$

In our implementation, the first-stage retriever returns $k_{\text{cand}} = 40$ candidate passages, from which the cross-encoder selects the final $k = 8$ passages provided to the generator. We use a large embedding model as the bi-encoder and a specialized embedding re-ranker as the cross-encoder, both belonging to the BGE retrieval model family [XLZ+24].

Cross-encoder re-ranking improves retrieval precision by refining the ranking of candidate passages returned by dense retrieval. However, it does not address the core challenge of multi-hop retrieval. Since all candidates are still retrieved using the original query, evidence required in later reasoning steps may never enter the candidate set in the first place. Addressing this limitation requires modifying the retrieval process itself, which motivates the query decomposition approach presented next.

2.1.4 Query decomposition

[Author: Tudor-Octavian Mihăiță]

Query decomposition addresses multi-hop questions by replacing a complex query with *a sequence of simpler sub-questions*, each targeting a specific reasoning step. The intuition is that retrieving evidence for several smaller information needs is often easier than retrieving

all supporting evidence with a single query. This is particularly relevant in multi-hop settings, where later pieces of evidence may have little lexical similarity to the original question.

Our implementation is inspired by Decomposed Prompting [KTF+22], using the language model itself to generate the sub-questions using **few-shot prompting**: in the given prompt with instructions for answering the question, the model receives a set of examples (question $\longrightarrow$ decomposition), and the model generalizes from those examples to decompose the new questions, without any additional training. Within this paradigm, we evaluate two variants: *static decomposition* and *iterative decomposition*.

Static decomposition. In static decomposition, the language model generates all sub-questions upfront. Each sub-question is retrieved independently, and the resulting rankings are combined using **Reciprocal Rank Fusion** (RRF) [CCB09]. The aggregation score is defined in Equation 4, where $\text{rank}_i(p)$ denotes the rank of passage p for the i-th sub-query and $c = 60$ is a smoothing constant, value chosen from the original paper. Since RRF operates on ranks rather than similarity scores, it is particularly suitable for combining retrieval results produced by different sub-queries.

$$\text{RRF}(p) = \sum_{i=1}^{n} \frac{1}{c + \text{rank}_i(p)}, \tag{4}$$

The overall procedure is summarized in Algorithm 2. Only two language-model calls are required: one for decomposition and one for final answer generation, conditioned on the aggregated relevant passages.

Algorithm 2 Static query decomposition

Input: Question q, corpus index $\mathcal{C}$, bi-encoder ϕ, language model LM, sub-query top-k size k_{sub}, final size k, decomposition prompt T_{decomp}
Output: Top-k passages

$\{q_1, \ldots, q_n\} \leftarrow \text{LM}(T_{\text{decomp}}(q))$ ▷ single LLM call: produce sub-questions
for $i = 1, \ldots, n$ **do**
  $\mathbf{q}_i \leftarrow \phi(q_i)$
  $\mathcal{P}_i \leftarrow \text{top-k}_{\text{sub}\, p \in \mathcal{C}}\ \cos(\mathbf{q}_i, \phi(p))$
end for
$\mathcal{P}_{\text{final}} \leftarrow$ top-k passages by RRF over $\{\mathcal{P}_1, \ldots, \mathcal{P}_n\}$
return $\mathcal{P}_{\text{final}}$

Static decomposition can improve retrieval coverage by expanding a question into multiple retrieval perspectives. However, the generated sub-questions remain independent of one another. Information discovered during one retrieval step cannot influence subsequent retrievals, which limits the method's effectiveness on questions requiring intermediate entities to be resolved before later evidence can be retrieved.

Iterative decomposition. Iterative decomposition addresses this limitation by introducing *intermediate reasoning steps* between retrieval operations. After retrieving evidence for a sub-question, the language model generates an intermediate answer, which is then used to rewrite subsequent sub-questions before retrieval. This allows newly discovered entities or facts to guide later retrieval steps.

The method is based on the formulation of **Self-Ask** [PZM+23] for using intermediate answers to guide reasoning, and is later expanded in **IRCoT** [TBKS23], which additionally interleaves retrieval with chain-of-thought reasoning. Therefore, our implementation first generates all sub-questions in a single decomposition step, and then progressively rewrites them as new information passed to the subsequent question, together with another retrieval

step for finding passages similar to the intermediate answers that the model may or may not detect.

Algorithm 3 summarizes the procedure. Compared to static decomposition, the method introduces additional language model calls for query rewriting and intermediate answer generation, allowing information gathered at earlier hops to influence later retrieval stages.

Algorithm 3 Iterative query decomposition

Input: Question q, corpus index $\mathcal{C}$, bi-encoder ϕ, language model LM, sub-query top-k size k_{sub}, final size k, intermediate top-k size k_{int}, prompts $T_{\text{decomp}}, T_{\text{rewrite}}, T_{\text{intermediate}}$
Output: Top-k passages

$\{q_1, \ldots, q_n\} \leftarrow \text{LM}(T_{\text{decomp}}(q))$
$\mathcal{A} \leftarrow [\,]$ ▷ intermediate answers accumulated so far
for $i = 1, \ldots, n$ **do**
  if $\mathcal{A}$ is non-empty **then**
    $q_i' \leftarrow \text{LM}(T_{\text{rewrite}}(q_i, \mathcal{A}))$ ▷ substitute resolved entities
  else
    $q_i' \leftarrow q_i$
  end if
  $\mathcal{P}_i \leftarrow \text{top-k}_{\text{sub}\,p \in \mathcal{C}}\ \cos(\phi(q_i'), \phi(p))$
  $a_i \leftarrow \text{LM}(T_{\text{intermediate}}(q_i, \mathcal{P}_i[:k_{\text{int}}]))$ ▷ generate intermediate answer
  if a_i is a useful answer (not a refusal) **then**
    append a_i to $\mathcal{A}$
  end if
end for
$\mathcal{P}_{\text{final}} \leftarrow$ top-k passages by RRF over $\{\mathcal{P}_1, \ldots, \mathcal{P}_n\}$
return $\mathcal{P}_{\text{final}}$

The increased flexibility comes at a higher computational cost. For a decomposition containing n sub-questions, iterative decomposition requires $2n + 1$ language-model calls, compared to two calls for static decomposition and a single call for naive retrieval. The method is therefore only beneficial if the improved retrieval quality offsets this additional inference cost.

Both decomposition variants depend on the language model producing valid sub-questions. When the decomposition output cannot be parsed or does not satisfy the expected format, the system falls back to naive retrieval on the original question. This ensures robust behavior even when decomposition is unsuccessful.

2.2 Dataset and Evaluation Corpus

This subsection describes the dataset used to evaluate the proposed retrieval methods and the corpus construction process employed in our experiments. We first introduce the MuSiQue benchmark [TBKS22], highlighting the characteristics that make it suitable for evaluating multi-hop question answering systems. We then describe how the evaluation corpus is constructed and prepared for retrieval.

2.2.1 The MuSiQue Benchmark

[Author: Alexandru Profir]

The experiments in this project are conducted on **MuSiQue** (Multi-hop Questions via Single-hop Composition) [TBKS22], a benchmark specifically designed to evaluate multi-hop question answering systems. Unlike earlier datasets such as HotpotQA [YQZ+18], MuSiQue was constructed to reduce shortcut solutions and ensure that answering a question requires combining information from multiple pieces of evidence.

Questions are created by composing several single-hop questions into reasoning chains of two, three, or four hops. As a result, answering a question requires retrieving and connecting evidence distributed across multiple documents rather than locating a single supporting passage. Each instance includes the question, the gold answer, the corresponding reasoning decomposition, and a set of candidate paragraphs annotated as either supporting evidence or distractors.

Questions are created by composing several single-hop questions into reasoning chains of two, three, or four hops. As a result, answering a question requires retrieving and connecting evidence distributed across multiple documents rather than locating a single supporting passage. Each instance includes the question, the gold answer, the corresponding reasoning decomposition, and a set of candidate paragraphs annotated as either supporting evidence or distractors.

Listing 1 presents an abbreviated example of a MuSiQue instance.

Listing 1: Abbreviated MuSiQue instance (2-hop). Distractor paragraphs and full paragraph text are shortened for brevity.

```
{
  "id": "2hop__123_456",
  "question": "Who is the spouse of the performer of Imagine?",
  "answer": "Yoko Ono",
  "answer_aliases": ["Ono Yoko"],
  "question_decomposition": [
    {
      "id": 0,
      "question": "Who performed Imagine?",
      "answer": "John Lennon",
      "paragraph_support_idx": 3
    },
    {
      "id": 1,
      "question": "Who is the spouse of #1?",
      "answer": "Yoko Ono",
      "paragraph_support_idx": 17
    }
  ],
  "paragraphs": [
    { "idx": 0,  "title": "...", "paragraph_text": "...",
      "is_supporting": false },
    ...
    { "idx": 3,  "title": "John Lennon",
      "paragraph_text": "John Lennon was an English singer-songwriter...",
      "is_supporting": true },
    ...
    { "idx": 17, "title": "Yoko Ono",
      "paragraph_text": "Yoko Ono is a Japanese multimedia artist...",
      "is_supporting": true }
  ]
}
```

2.2.2 Evaluation corpus construction

[Author: Alexandru Profir]

The experiments are conducted on a stratified sample of 500 questions drawn from the MuSiQue development split. Evaluating retrieval-intensive methods on the full development set is computationally expensive, particularly for approaches that require multiple language-model calls per question. Following common practice in the multi-hop QA literature [TBKS23], we therefore evaluate on a representative subset rather than the complete benchmark.

The sample contains 200 two-hop questions, 200 three-hop questions, and 100 four-hop questions. This distribution intentionally oversamples deeper reasoning chains relative to their frequency in the original dataset, ensuring sufficient examples for stable per-hop evaluation. Sampling is performed uniformly within each hop category using a fixed random seed (`seed = 42`) to ensure reproducibility. Table 1 summarizes the resulting evaluation set.

The retrieval corpus is constructed by pooling all candidate paragraphs associated with the sampled questions and removing duplicate passages. Since each MuSiQue question is accompanied by approximately 20 candidate paragraphs, the resulting corpus contains roughly 10,000 unique passages after deduplication. Passages are identified using a SHA-1 hash of their textual content, providing stable identifiers while ensuring that identical passages appearing across multiple questions are indexed only once.

Unlike the original MuSiQue setup, **retrieval is performed over the entire pooled corpus** rather than over the candidate paragraphs associated with an individual question. This creates a more realistic open-domain retrieval setting, where supporting evidence must compete against thousands of unrelated passages. Consequently, retrieval metrics such as Recall@k and Mean Reciprocal Rank (MRR) provide a more meaningful measure of retrieval quality than they would in a small per-question candidate set.

The same corpus and vector index are used for all evaluated methods. As a result, differences in performance can be attributed solely to the retrieval strategy itself rather than to variations in the underlying knowledge source.

	2-hop	**3-hop**	**4-hop**
Full dev split (questions)	1,252 (51.8%)	760 (31.4%)	405 (16.8%)
Evaluation sample (questions)	200	200	100
Sample fraction	16.0%	26.3%	24.7%
Gold passages per question	2	3	4
Pooled retrieval corpus (combined across all sampled questions)			
Unique passages	∼10,000		
Average paragraph length	83 words (median 72)		

Table 1: MuSiQue development split, evaluation sample, and pooled retrieval corpus. Sample fractions denote the proportion of the available dev questions at each hop count selected for evaluation.

2.3 Application

This subsection presents the application developed to implement and evaluate the retrieval methods described previously. We first provide an overview of the system architecture and the design decisions that enable a fair comparison between retrieval strategies. We then describe the interactive interface used to inspect, compare, and benchmark the behavior of the implemented methods.

2.3.1 Architectural overview

[Author: Tudor-Octavian Mihăiță]

The application is designed as a benchmark tool for the described RAG approaches, in the context of QA tasks, by prompting a LLM backbone. The followed architectural principle is to only switch the retrieval strategy in QA pipeline. All retrieval approaches operate on the same document corpus, vector index, embedding model, and language model, ensuring that differences in performance can be attributed solely to the retrieval mechanism itself.

The system is organized into three logical layers. The **data layer** contains the MuSiQue evaluation sample, the pooled retrieval corpus, and the vector index built from that corpus. These resources are prepared offline and remain fixed during evaluation. The **retrieval layer** implements the different retrieval strategies evaluated in this project, exposing a common interface that accepts a question and returns a ranked set of passages. The **generation layer** is responsible for answer generation, consuming the retrieved context and producing the final response through a language model.

A key design decision is the use of a pluggable retrieval interface. The pipeline remains unchanged regardless of the selected retrieval method, allowing retrievers to be exchanged without modifying any other component. This design enables both the offline evaluation framework and the interactive application to share the same retrieval and generation pipeline, ensuring consistency between reported experimental results and user-facing behavior.

The application separates offline and online processing stages. Offline stages include dataset preparation, corpus construction, embedding generation, and index creation. Online stages consist of per-query retrieval and answer generation. Figure 1 illustrates the overall architecture and the interaction between these components.

To support flexible deployment, the generation layer abstracts the underlying language model backend. The current implementation supports both local inference through **Ollama** [Oll24] and remote inference through the **OpenAI API**, while exposing a uniform interface to the rest of the system.

2.3.2 Interactive benchmarking interface

[Author: Tudor-Octavian Mihăiță]

In addition to the offline evaluation framework, the application provides a **web-based interface implemented with Streamlit** [Str24]. The interface complements aggregate evaluation metrics by allowing retrieval behavior to be inspected on individual examples. This makes it useful both as a pedagogical tool, helping users understand how different retrieval strategies operate, and as a development aid for analyzing retrieval and generation failures.

The interface supports three modes. In **Chat** mode, users interact with a selected retrieval method as a standard question-answering system. Alongside the generated answer, the interface exposes the retrieved passages and, when applicable, the decomposition outputs and intermediate reasoning steps used by the method.

Compare mode executes the same question using all implemented retrieval strategies and presents the resulting answers side-by-side. This enables direct qualitative comparison of retrieval behavior and answer quality, making differences between methods immediately visible.

Finally, **Benchmark** mode evaluates methods on questions sampled from the MuSiQue dataset. In addition to the generated answers, the interface reveals the gold answer and supporting passages provided by the dataset annotations. Retrieved passages are compared against the gold evidence, allowing users to inspect retrieval successes and failures at the level of individual questions.

All three modes operate on the same retrieval and generation pipeline described in Section 2.3.1. The underlying retrievers, vector index, and language-model backend remain unchanged; only the way results are presented differs between modes. This ensures consistency between the interactive application and the offline experiments reported later in the report.

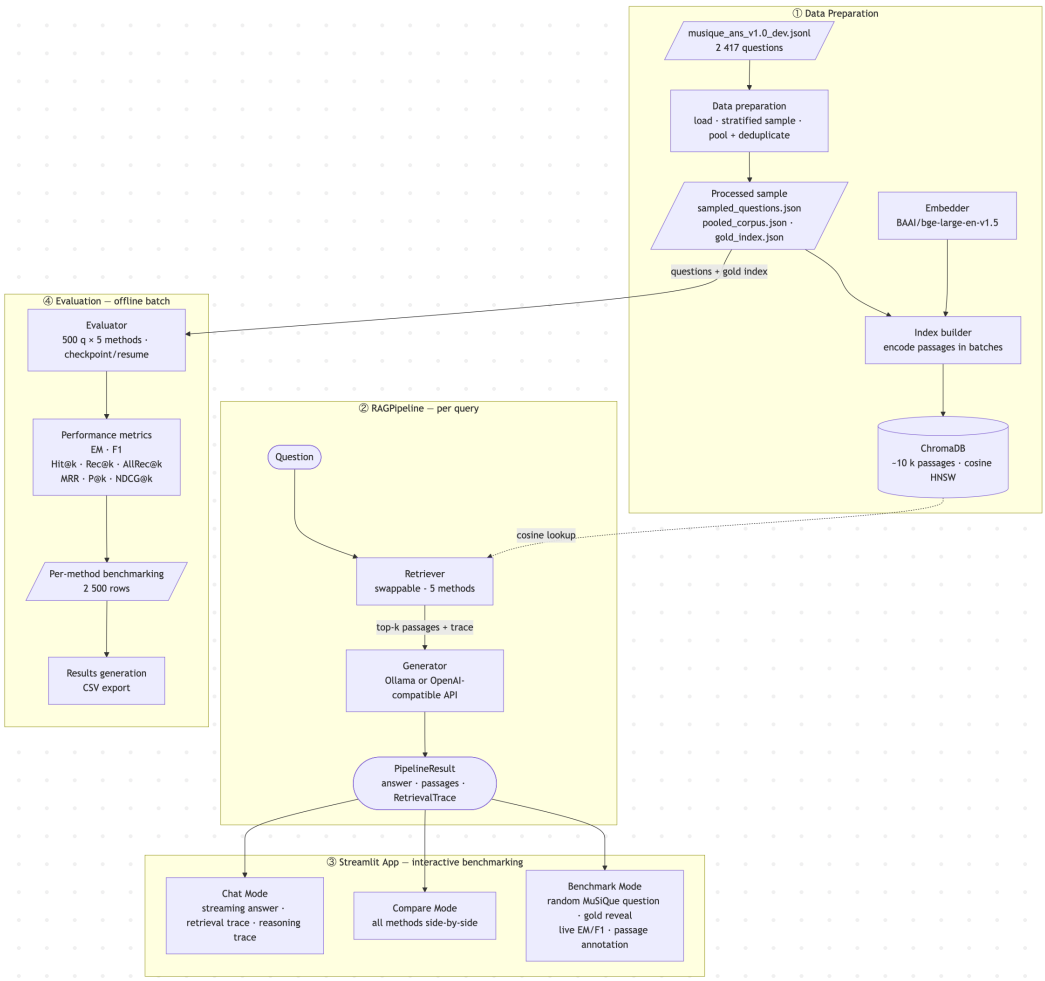

Figure 1: System architecture. A shared retrieval-generation pipeline operates over a common corpus and vector index, while different retrieval strategies can be evaluated through either the interactive application or the offline benchmarking framework.

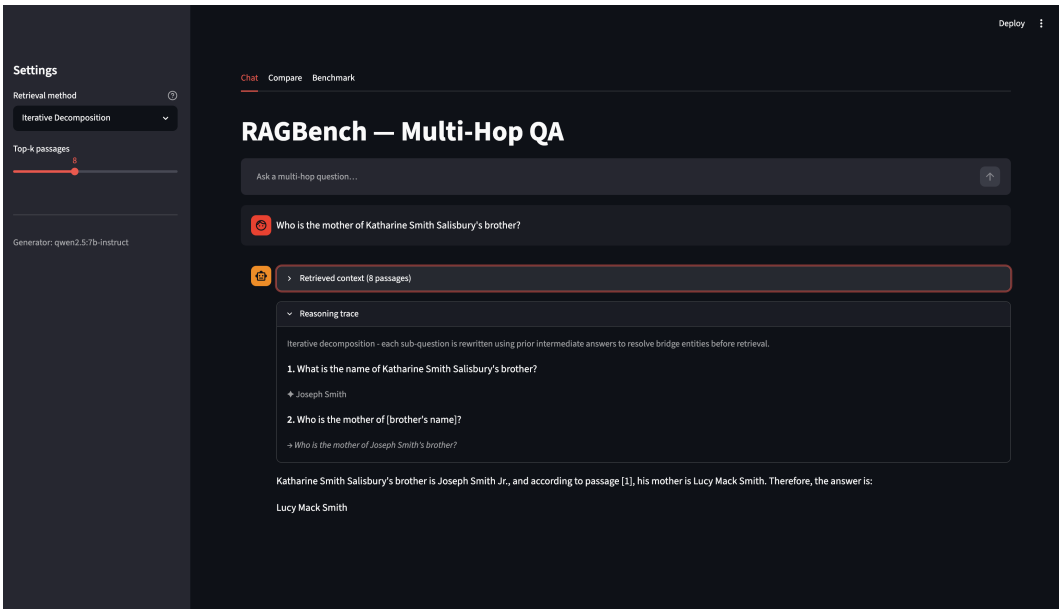

Figure 2: Chat mode. Users can query a selected retrieval method and inspect the retrieved passages and reasoning trace behind the generated answer.

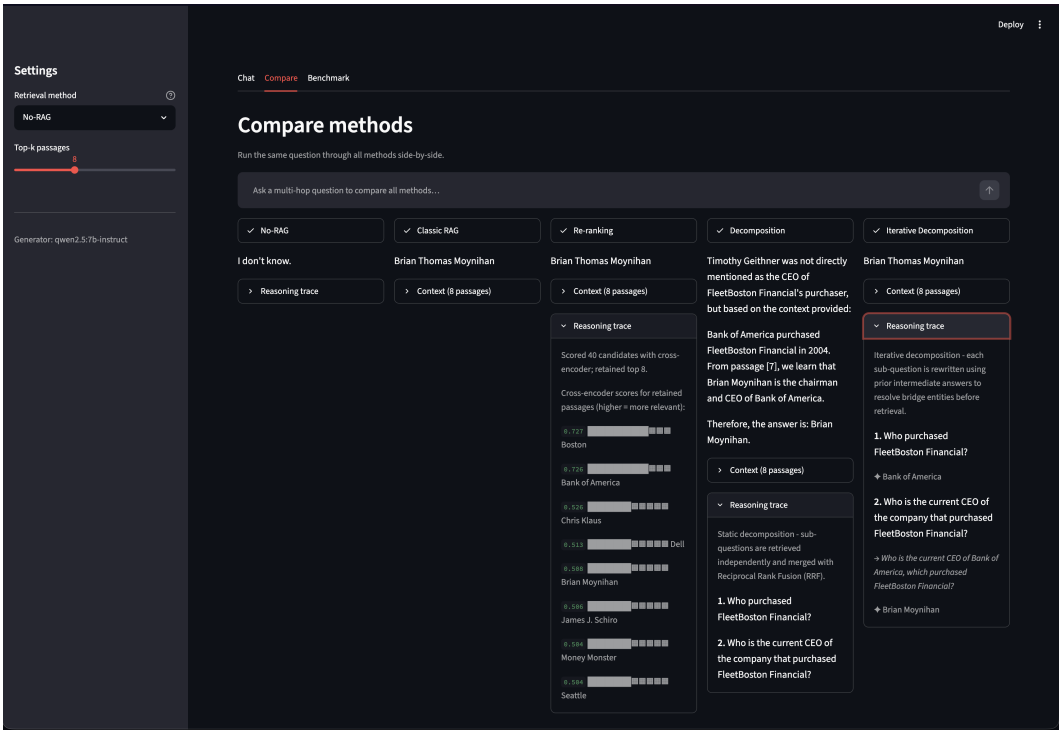

Figure 3: Compare mode. The same question is executed across all retrieval methods, enabling side-by-side comparison of answers, retrieved evidence, and reasoning traces.

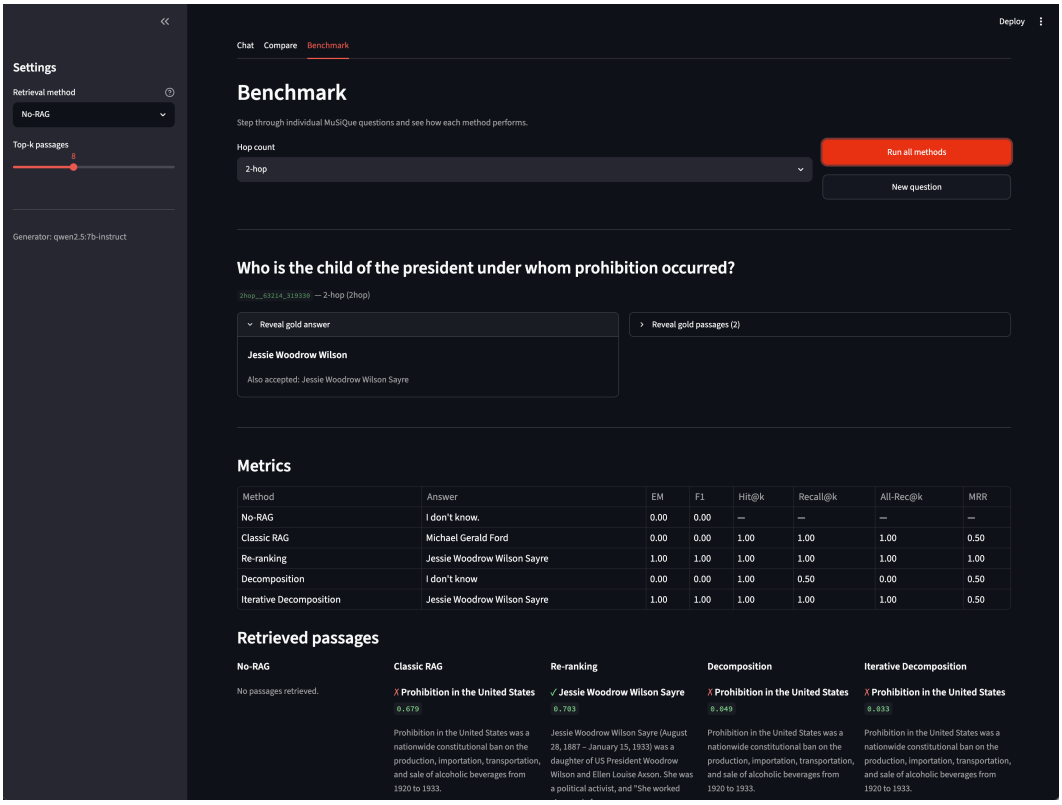

Figure 4: Benchmark mode. Questions from the MuSiQue evaluation set are used to compare method outputs against gold answers and supporting passages.

3 Implementation Details

This section describes the implementation of the architecture presented in Section 2.3. We first introduce the technology stack and then discuss the main components of the system, including data preparation, retrieval, generation, and application orchestration. The focus is placed on implementation decisions that influence the behavior, extensibility, or performance of the system rather than on low-level code details.

3.1 Technology stack

[Author: Tudor-Octavian Mihăiță]

The application is implemented in Python 3, with dependencies managed through the `uv` package manager. The technology stack was selected to support a modular architecture in which individual components, such as the embedding model, vector store, or language model backbone, can be replaced with minimal changes to the surrounding system.

Role	**Library**	**Purpose**
Vector store	`chromadb`	Persistent dense-vector index with cosine similarity search.
Embeddings	`sentence-transformers`	Bi-encoder and cross-encoder model loading and inference.
Tensor backend	`torch`	Underlying compute backend; supports CPU, CUDA, and Apple MPS.
LLM (local)	`ollama`	Native client for Ollama-served language models.
LLM (remote)	`httpx`	HTTP client for OpenAI-compatible chat-completion endpoints.
User interface	`streamlit`	Web-based interactive application.
Numerical utilities	`numpy`	Embedding arrays, similarity computation.
Configuration	`pydantic-settings`	Typed configuration with environment-variable overrides.
Logging	`loguru`	Structured logging across scripts and pipelines.
CLI rendering	`rich`	Formatted tables for evaluation output.

Table 2: Principal libraries used in the implementation, organized by role.

Two implementation choices are particularly important. First, **ChromaDB** [Chr24] is used as a persistent local vector store. Given the relatively small size of the evaluation corpus (approximately 10,000 passages), a lightweight embedded database provides sufficient performance while avoiding the operational complexity of a dedicated vector database service. Separate indexes are maintained for each embedding model, allowing multiple retrieval configurations to coexist without rebuilding previously generated embeddings.

Second, answer generation is abstracted behind a unified generator interface. This layer supports both local inference through **Ollama** [Oll24] and remote inference through **OpenAI-compatible APIs**, enabling the same retrieval pipeline to operate independently of the underlying language model deployment strategy. As a result, the evaluation framework and interactive application share the same generation component and differ only in how results

are presented to the user.

3.2 Data preparation and corpus indexing

[Author: Alexandru Profir]

Data preparation consists of two stages: corpus construction and vector indexing. First, a stratified sample of MuSiQue questions is selected according to the procedure described in Section 2.2.2. The candidate paragraphs associated with these questions are then pooled into a single retrieval corpus and deduplicated to ensure that passages appearing in multiple questions are indexed only once. During this process, a mapping between questions and their supporting passages is preserved, allowing retrieval metrics to be computed later during evaluation.

Once the corpus has been constructed, all passages are converted into dense vector representations using a sentence embedding model and stored in a persistent ChromaDB index. The default embedding model used throughout the project is `BAAI/bge-large-en-v1.5` [XLZ+24], a high-performing retrieval model from the BGE family. The resulting vector index is shared by all evaluated retrieval methods, ensuring that performance differences arise from the retrieval strategy itself rather than from variations in the underlying corpus representation.

A practical implementation detail concerns query and passage formatting. Several modern embedding models expect *task-specific prefixes* to be added to the input text before encoding. For example, BGE models require a retrieval-oriented instruction to be prepended to queries, while other embedding families use different conventions. The embedding component therefore applies model-specific prefixes automatically, ensuring that embeddings are generated according to the recommendations of the corresponding model authors.

To support experimentation with multiple embedding models, indexes are stored independently and can be rebuilt without affecting previously generated representations. This allows alternative embedding configurations to be evaluated while preserving the reproducibility of existing experiments.

3.3 Generator backbone

[Author: Tudor-Octavian Mihăiță]

The generation component is implemented as a backend-independent abstraction that provides a uniform interface for all retrieval methods. This design allows the retrieval pipeline to operate independently of the underlying language model, making it possible to switch between local and remotely hosted models without modifying the rest of the system.

The primary generation backend used in this project is Ollama [Oll24], which enables local inference on commodity hardware. Benchmarking experiments were conducted using the **Qwen 7B instruct** [Qwe24] and **Gemma 4 31B** [Goo25] models, both of which provide a favorable balance between computational requirements and reasoning capability for multi-hop question answering. Due to hardware limitations, the smaller Qwen model is used for the local inference demonstration of the RAG methods, while the benchmark results are generated using the more capable Gemma model, given its higher number of parameters and therefore greater usage of retrieved context. The evaluation model offers an OpenAI-compatible API, allowing the same retrieval pipeline to be evaluated with externally hosted language model executed remotely.

A key objective of the implementation is ensuring that retrieval strategies are compared under identical generation conditions. Consequently, all methods share the same generator configuration and prompting scheme. Furthermore, generation is performed with a temperature of 0.0 throughout all experiments, ensuring deterministic outputs and eliminating sampling variability as a source of performance differences.

By abstracting the generation backend behind a common interface, the system isolates retrieval from generation concerns. This allows alternative language models to be incorporated with minimal changes while preserving a consistent evaluation framework.

3.4 Retriever implementation

[Author: Alexandru Profir]

The retrieval layer is implemented around a common retriever interface that accepts a question and returns a ranked list of passages. This abstraction allows all retrieval methods to be integrated into the same generation pipeline while exposing method-specific diagnostic information used by the evaluation framework and interactive interface. Figure 5 shows this implementation choice in the context of the system-level class diagram.

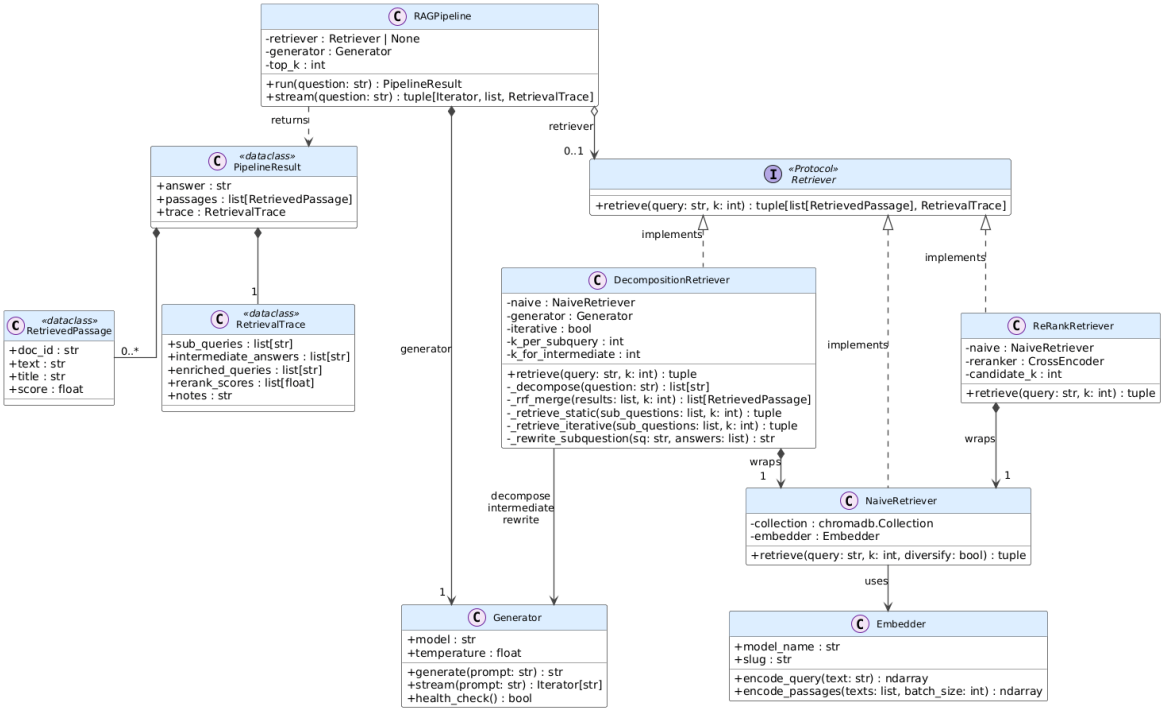

Figure 5: Class diagram of the retrieval and generation pipeline. Different retrieval strategies implement a common `Retriever` interface and are composed with a shared `Generator` through the `RAGPipeline`.

A key design objective is ensuring that retrieval strategies can be compared under identical conditions. Consequently, all retrievers operate on the same document corpus, vector index, embedding model, and generator. The only difference between methods lies in how candidate passages are selected and ranked.

The naive retriever serves as the baseline implementation. Given a query, it retrieves the top-k passages from the vector index using cosine similarity between dense embeddings. To improve evidence diversity for multi-hop questions, a lightweight diversification step limits the number of passages originating from the same source document, reducing the tendency of dense retrieval to return multiple highly similar passages from a single article.

The re-ranking retriever extends this baseline through a two-stage retrieval process. A larger candidate set is first retrieved using dense retrieval and then re-ordered using the `BAAI/bge-reranker-base` cross-encoder [XLZ+24]. This implementation directly follows the retrieve-then-rerank procedure described in Section 2.1.3, improving ranking precision while retaining the efficiency of vector search.

The decomposition retriever implements both the static and iterative query decomposition

strategies presented in Section 2.1.4. In both variants, the language model first decomposes the original question into a sequence of sub-questions. Static decomposition retrieves evidence independently for each sub-question and combines the resulting rankings using Reciprocal Rank Fusion (Equation 4). Iterative decomposition additionally incorporates intermediate answers into subsequent retrieval steps, allowing information discovered at earlier hops to influence later queries. To ensure robustness, malformed decompositions automatically trigger a fallback to the baseline retrieval strategy.

Retrieval outputs are subsequently passed to a shared pipeline component responsible for orchestrating retrieval and generation. Given a question, the pipeline retrieves supporting passages, constructs the final prompt, and invokes the language model to generate an answer. The no-RAG baseline is implemented as a special case in which retrieval is skipped entirely and the generator receives only the original question.

This separation between retrieval and generation is central to the experimental design. Because all pipelines share the same corpus, embedding model, and language model configuration, differences in performance can be attributed directly to the retrieval strategy rather than to variations elsewhere in the system.

3.5 Prompt engineering techniques

[Author: Tudor-Octavian Mihăiță]

Prompt engineering plays an important role in both answer generation and query decomposition. To ensure a fair comparison between retrieval methods, all approaches share the same prompt templates whenever they perform the same task. Table 3 summarizes the prompts used throughout the system.

The final answer-generation prompts are responsible for producing concise factual answers, with separate variants for the RAG and no-RAG settings. The decomposition prompt is used by both query decomposition methods to transform a complex question into a sequence of self-contained sub-questions, an example of which is shown in Listing 2. Iterative decomposition additionally relies on an intermediate-answer prompt and a query-rewriting prompt, which allow information discovered at earlier reasoning steps to influence later retrieval stages.

Listing 2: Example output produced by the decomposition prompt.

```
Question:
In which city was the director of
'The Dark Knight' born?

1. Who directed 'The Dark Knight'?
2. In which city was Christopher Nolan born?
```

The prompt templates were refined through small-scale testing on representative MuSiQue questions and then kept fixed throughout all experiments. This ensures that differences in performance are attributable to retrieval strategy rather than to prompt variation. For reproducibility, the same prompts are shared by both the offline evaluation framework and the interactive application.

4 Experiments and Results

This section presents the experimental evaluation of the retrieval methods introduced in Chapter 2. We first describe the evaluation setup, including the dataset, models, and metrics

Prompt	Used by	Purpose
`RAG_PROMPT`	All RAG methods	Generates the final answer using the retrieved context. Encourages concise factual responses grounded in the retrieved passages.
`NO_RAG_PROMPT`	No-RAG baseline	Generates answers without external context while allowing explicit uncertainty when knowledge is unavailable.
`DECOMPOSITION_PROMPT`	Static and iterative decomposition	Breaks a multi-hop question into a sequence of self-contained sub-questions.
`INTERMEDIATE_PROMPT`	Iterative decomposition	Produces an intermediate answer for a single reasoning hop from the retrieved evidence.
`REWRITE_PROMPT`	Iterative decomposition	Rewrites later sub-questions using facts discovered in previous hops.

Table 3: Prompt templates used throughout the system.

used throughout the experiments. We then compare the retrieval and question-answering performance of the implemented methods and analyze their behavior across different reasoning depths.

4.1 Experimental setup

[Author: Alexandru Profir]

All five methods evaluated in this project, No-RAG, Classic RAG, Re-ranking, Static Decomposition, and Iterative Decomposition, are tested on the 500-question MuSiQue sample described in Section 2.2.2. The sample contains 200 two-hop questions, 200 three-hop questions, and 100 four-hop questions.

To ensure a fair comparison, all methods share the same retrieval corpus, embedding model, generator, and prompt templates. The retrieval strategy is therefore the only experimental variable.

The generator used throughout the experiments is `gemma4:31b` [Goo25], executed remotely via a OpenAI backend API at a temperature of 0.0. Dense retrieval is performed using the `bge-large-en-v1.5` embedding model [XLZ+24], while re-ranking uses the `bge-reranker-base` cross-encoder. Because generation with a temperature set to 0 is considered deterministic, differences in answer quality can be attributed directly to differences in the retrieved evidence rather than to sampling variability.

Table 4 summarizes the configuration shared across all experiments.

Performance is evaluated using both generation and retrieval metrics. For answer generation, we report **Exact Match** (EM) and **token-level F1**, computed against the gold answer and its aliases using the standard normalization procedure introduced by the **Stanford Question Answering Dataset** (SQuAD) [RZLL16], which lowercases text and removes punctuation, articles, and extra whitespace. For retrieval, we report specific metrics such as Hit@k, Recall@k, **All-Recall@**k, **Mean Reciprocal Rank (MRR)**, **Precision@**k, and NDCG@k, using the gold supporting passages provided by MuSiQue.

Among these metrics, All-Recall@k is particularly important for multi-hop question answering. Unlike Recall@k, which measures the proportion of supporting passages retrieved, All-Recall@k is only satisfied when **every supporting passage is present in the retrieved context**. Since multi-hop questions typically require all evidence pieces to recon-

Component	Setting	Value
Generator	Model	`gemma3:31b`
	Temperature	0.0
	Backend	Ollama
Embedder	Model	`BAAI/bge-large-en-v1.5`
	Dimension	1024
Re-ranker	Model	`BAAI/bge-reranker-base`
	Candidate pool	40
Retrieval	Final top-k	8
	Sub-query top-k (decomposition)	10
	Intermediate top-k (iterative)	4
Dataset	Sample size	500
	Sampling seed	42
	Hop stratification	200 / 200 / 100 (2/3/4-hop)

Table 4: Experimental configuration shared across all evaluation runs.

struct the reasoning chain, this metric provides a stronger indication of whether retrieval is sufficient to support answer generation.

4.2 Results and discussion

[Author: Alexandru Profir]

The full experimental results are presented across three tables: aggregate metrics averaged over all sampled questions (Table 5), generation metrics stratified by hop count (Table 6), and retrieval metrics stratified by hop count (Table 7). We discuss the principal findings of each in turn, then turn to qualitative analysis on individual questions and to the limitations of the evaluation metrics themselves.

Method	EM	F1	Hit@k	Rec@k	AllRec@k	MRR	P@k	NDCG@k
No-RAG	0.008	0.037	—	—	—	—	—	—
Classic RAG	0.122	0.187	0.936	0.587	0.242	0.765	0.197	0.568
Re-ranking	0.204	0.301	0.938	0.597	0.232	**0.804**	0.203	0.588
Decomposition	0.248	0.326	**0.964**	0.618	0.272	0.725	0.206	0.551
Iterative Decomposition	**0.356**	**0.426**	0.954	**0.709**	**0.438**	0.681	**0.236**	**0.596**

Table 5: Aggregate metrics averaged over all 500 sampled questions. Best results in each column are shown in bold. No-RAG does not retrieve and therefore has no retrieval metrics.

Retrieval is essential for multi-hop QA. The No-RAG baseline achieves only 0.008 EM and 0.037 F1, indicating that the generator alone is largely unable to solve MuSiQue questions from parametric memory. Even Classic RAG improves F1 to 0.187, demonstrating the importance of external retrieval for multi-hop reasoning.

Reasoning depth remains the primary challenge. As shown in Table 6, all methods degrade substantially as hop count increases. Classic RAG drops from 0.241 F1 on 2-hop questions to 0.117 on 4-hop questions, while Iterative Decomposition falls from 0.580 to 0.153. The retrieval metrics explain this trend: although all methods retrieve at least one relevant passage in most cases (Hit@$k > 0.92$ across all hop counts), retrieving the complete

supporting evidence becomes increasingly difficult. For example, Classic RAG's All-Recall@k decreases from 0.435 on 2-hop questions to only 0.020 on 4-hop questions.

Method	2-hop		3-hop		4-hop	
	EM	F1	EM	F1	EM	F1
No-RAG	0.020	0.042	0.000	0.033	0.000	0.036
Classic RAG	0.175	0.241	0.100	0.167	0.060	0.117
Re-ranking	0.265	0.363	0.205	0.321	0.080	0.137
Decomposition	0.375	0.460	0.195	0.289	0.100	0.130
Iterative Decomposition	**0.490**	**0.580**	**0.335**	**0.409**	**0.130**	**0.153**

Table 6: Generation metrics (EM and F1) stratified by hop count.

Method	Hit@k	Rec@k	AllRec@k	MRR	P@k	NDCG@k
2-hop						
Classic RAG	0.925	0.680	0.435	**0.820**	0.170	0.654
Re-ranking	0.925	0.642	0.360	0.785	0.161	0.611
Decomposition	0.960	0.735	0.510	0.772	0.184	0.647
Iterative Decomp.	**0.960**	**0.838**	**0.715**	0.782	**0.209**	**0.727**
3-hop						
Classic RAG	0.955	0.582	0.160	0.755	0.218	0.556
Re-ranking	0.955	0.630	0.215	**0.828**	0.236	**0.621**
Decomposition	**0.970**	0.588	0.160	0.715	0.221	0.520
Iterative Decomp.	0.960	**0.688**	**0.345**	0.630	**0.258**	0.546
4-hop						
Classic RAG	0.920	0.412	0.020	0.677	0.206	0.417
Re-ranking	0.930	0.440	0.010	**0.794**	0.220	**0.476**
Decomposition	**0.960**	0.443	0.020	0.650	0.221	0.422
Iterative Decomp.	0.930	**0.492**	**0.070**	0.585	**0.246**	0.435

Table 7: Retrieval metrics stratified by hop count. Best results are shown in bold and second-best results are underlined.

Re-ranking provides a strong low-cost improvement. Cross-encoder re-ranking consistently outperforms Classic RAG, increasing overall F1 from 0.187 to 0.301. The largest gains appear on 3-hop questions, where F1 improves from 0.167 to 0.321. The improvement is reflected in the higher MRR values, indicating that relevant passages are ranked closer to the top of the retrieval list. However, re-ranking cannot recover passages that never enter the candidate pool, which limits its effectiveness on the most difficult 4-hop questions.

Iterative decomposition performs best overall. Iterative Decomposition achieves the strongest results across nearly all metrics, reaching 0.356 EM and 0.426 F1 overall. Its largest advantage appears in All-Recall@k, where it substantially outperforms all other methods. This suggests that resolving intermediate entities before issuing later retrievals successfully addresses one of the main weaknesses of dense retrieval.

A representative example is the question *"Who was married to the creator of Ruby Loftus Screwing a Breech Ring?"*. Classic RAG, Re-ranking, and Static Decomposition fail to retrieve the passage about Harold Knight and therefore cannot answer correctly. Iterative Decomposition first identifies *Laura Knight* as the creator of the painting and then rewrites the

next sub-question using this intermediate result, enabling retrieval of the correct supporting passage and ultimately the correct answer.

The 4-hop regime remains largely unsolved. Even the strongest method achieves only 0.153 F1 on 4-hop questions, while no method exceeds 0.07 All-Recall@k. These results suggest that the primary bottleneck is no longer ranking quality but the difficulty of retrieving all supporting evidence within a fixed retrieval budget. More advanced retrieval-reasoning strategies, such as IRCoT-style retrieval interleaving, may be required to address this limitation.

Metric limitations. Several examples reveal shortcomings of EM and token-level F1. **Answers that contain the correct entity together with additional valid information are often penalized despite being factually correct**, while partially overlapping but incorrect answers can receive non-zero scores. For example, Iterative Decomposition correctly identifies the gold answer *"Citrus Heights"* for one question but additionally lists neighboring cities, resulting in a reduced F1 score despite providing useful information. Future work could incorporate semantic evaluation methods such as LLM-as-a-judge scoring or RAGAS-based evaluation [EJEAS24] to better capture answer quality.

Summary. The experiments support four conclusions. First, retrieval substantially improves performance over parametric generation alone. Second, cross-encoder re-ranking is a simple and effective enhancement to dense retrieval. Third, static decomposition is particularly beneficial on simpler multi-hop questions but struggles as reasoning depth increases. Finally, iterative decomposition is the most effective training-free retrieval strategy evaluated, largely because it improves the probability of retrieving the complete evidence chain required for multi-hop reasoning.

5 Conclusions and Future Work

[Author: Tudor-Octavian Mihăiţă]

This project investigated the impact of training-free retrieval-side techniques on open-domain multi-hop Question Answering. We implemented and evaluated Classic RAG, Cross-Encoder Re-ranking, Static Query Decomposition, and Iterative Query Decomposition on a stratified sample of 500 questions from the MuSiQue benchmark, while keeping the corpus, generator, embedding model, and prompts fixed. An interactive Streamlit application was also developed to support qualitative comparison of the evaluated methods.

The results show that **retrieval is essential for multi-hop QA**, with all RAG methods significantly outperforming the No-RAG baseline. Re-ranking provided a strong low-cost improvement over naive retrieval, while query decomposition was particularly beneficial on multi-hop questions. Among all methods, **Iterative Decomposition achieved the best overall performance** by resolving intermediate entities before subsequent retrieval steps, substantially improving complete-evidence retrieval. Nevertheless, **performance decreased sharply as hop count increased**, and all methods struggled on 4-hop questions, highlighting the difficulty of retrieving an entire reasoning chain within a fixed retrieval budget, and further highlighting the reasoning limitations of a language model despite additional correct context sometimes.

Future work could explore **semantic evaluation methods** such as LLM-as-a-judge scoring or RAGAS, stronger generator models with higher context windows and parametric memory, and **more robust retrieval approaches** that attempt to overcome the limitations of static retrieval, such as self-reflective methods that require additional training, or graph-based methods for entity and relation modeling. More broadly, the experiments demonstrate

that retrieval strategy plays a critical role in multi-hop question answering, but also that retrieval-side improvements alone are insufficient to fully address the challenges posed by deep reasoning tasks.

Usage of Generative AI

Acknowledgement: This work is the result of our own activity, and we confirm that we have neither given, nor received unauthorized assistance for this work. We declare that during the preparation of this work, the authors used generative AI or automated tools, specifically Anthropic's Claude, in drafting this document. After using the tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[CCB09] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.

[Chr24] Chroma. Chroma: The open-source ai application database. `https://github.com/chroma-core/chroma`, 2024. Software.

[EJEAS24] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated Evaluation of Retrieval Augmented Generation . In Nikolaos Aletras and Orphee De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, March 2024. Association for Computational Linguistics.

[Goo25] Google DeepMind. Gemma-4-31B. `https://huggingface.co/google/gemma-4-31B`, 2025. Model card.

[KOM+20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense Passage Retrieval for Open-Domain Question Answering . In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics.

[KTF+22] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks . In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[NC19] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.

[Oll24] Ollama Contributors. Ollama: Get up and running with large language models locally. `https://github.com/ollama/ollama`, 2024. Software, version 0.24.0.

[PZM+23] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.

[Qwe24] Qwen Team. Qwen2.5-7B-Instruct. `https://huggingface.co/Qwen/Qwen2.5-7B-Instruct`, 2024. Model card.

[RG19] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[RZLL16] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392, 2016.

[Str24] Streamlit Inc. Streamlit: The fastest way to build and share data apps. `https://streamlit.io`, 2024. Software.

[TBKS22] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

[TBKS23] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 10014–10037, 2023.

[XLZ+24] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed Resources For General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '24, page 641–649, New York, NY, USA, 2024. Association for Computing Machinery.

[YQZ+18] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.